

# Earned Autonomy

Conditioning Agent Trust on Delivered Outcomes, Not Task Accuracy

Andrew Zigler

July 2026

## Executive summary

Enterprises have adopted autonomous software agents faster than they have learned to tell whether the agents are delivering. The dominant answer to that trust problem, published under names like earned, graduated, and progressive autonomy, conditions an agent’s authority on how accurately it performs its tasks. This paper argues that the choice of variable is the mistake. Autonomy conditioned on task accuracy re-creates at the trust layer the same clean-record trap the audit layer already has: a fully authorized, correctly typed, cleanly traced call can open a defective change while every record reads clean, and an accuracy-graded ladder promotes a workflow at the moment its output draws the least scrutiny. Delivered outcome — whether an agent’s change held in production — is the only conditioning variable that closes the loop, and the Model Context Protocol revision finalizing on 2026-07-28 is the first standard substrate on which that condition is enforceable per action. The paper walks what the release makes legible and enforceable, defines a *delivery evidence layer* that joins governed actions to their delivered outcomes, and describes earned autonomy: a rolling trust cadence, modeled on the standard’s own governance by measurement, that grants and revokes agent authority on evidence.

## 1 The question that predates the spec

Organizations have poured budget, headcount, and roadmap into autonomous software work, and most of them still cannot say whether it paid off. MIT Media Lab’s Project NANDA, surveying roughly three hundred enterprise generative-AI initiatives, found that about 95% showed no measurable P&L return [10]. That figure has many causes, and the most conservative one is worth stating plainly: a return you cannot attribute is a return you cannot claim. At minimum, the 95% is an attribution failure.

Adoption is not the constraint. Stanford’s AI Index put enterprise AI adoption at 78% in its 2025 edition and 88% a year later [17, 18]. The tools are in the building. What remains unanswered is whether the autonomous work they produce is delivering, and for whom.

Scale is what makes the gap urgent. A single developer reviewing a single assistant’s suggestions is a manageable trust problem. A fleet of agents opening changes across dozens of repositories, at a rate no human reads line by line, is not. Trust at that scale has to be granted by policy rather than by inspection, and policy needs a signal to run on. The governance question here is older than any protocol. Before a wire format made agent activity legible, leaders already needed to know which autonomous work had earned more scope and which had not. The Model Context Protocol revision finalizing on 2026-07-28 does not create that question. It changes what an organization can answer it with.

The question also already has a named answer in the field, and that answer is worth taking seriously before improving on it. Earned autonomy, graduated autonomy, and progressive autonomy all describe the same ladder: an agent starts under human review, and as it performs, it earns the right to act with less oversight. Matthias Roder’s earned-autonomy gradient formalizes the rungs [16]; a growing set of platform vendors ship progressive-autonomy controls [1]; the Digital

Apprentice line of work models the same apprenticeship arc [5]; McKinsey and AWS both frame agent trust as something conditioned and revoked over time [3, 9]. This ladder is published across vendors and research, and it is converging on a shared shape.

Every published version conditions autonomy on the same class of signal: how well the agent performed its task. Extraction accuracy, override rate, tool-call correctness. That choice of variable is the one thing worth revisiting, because it is the same variable that lets a clean record hide a bad outcome. The rest of this paper argues for a different conditioning variable, and shows why the 2026-07-28 spec is the first standard substrate on which it can be enforced.

## 2 What the spec makes legible and enforceable

The 2026-07-28 revision is the largest change to the protocol since its launch, and for a governance argument its importance is specific. After this revision, every agent action is at once a well-formed record and a policy decision point. Five properties of the release, taken together, produce that result. None of them is invented here; each is a mechanism the release already ships, and the point is what they add up to.

**Identity-bound.** Every request now carries the client’s identity in its `_meta` block rather than establishing it once per session: the `io.modelcontextprotocol/clientInfo` field travels with the call itself (SEP-2575) [12]. On the authorization side, the release hardens OAuth so a client must validate a present issuer (`iss`) parameter against the recorded authorization server before it redeems a code, closing a class of mix-up attack (SEP-2468) [12]. An action arrives attributable to an authenticated principal and a named client implementation.

**Typed.** Tool inputs and outputs are lifted to full JSON Schema 2020-12 (SEP-2106), so the shape of every call and its result is declared and checkable rather than conventional [12]. A record whose payloads validate against a schema is a record a downstream system can parse without guessing at its structure.

**Traceable.** The release documents W3C Trace Context conventions — `traceparent`, `tracestate`, and `baggage` in `_meta` (SEP-414) — so a single action correlates across SDKs, gateways, and any OpenTelemetry backend [12]. The deprecation of the old Logging primitive points the same way: diagnostic logging moves to `stderr` and OpenTelemetry (SEP-2577) [11]. Observability belongs to the tracing plane, not to a bespoke protocol message.

**Enforceable.** Because the release removes protocol sessions (SEP-2567), any request can land on any server replica, and authorization is therefore evaluated on every call rather than inherited from a session established once [2]. The release also requires routing headers, `Mcp-Method` and `Mcp-Name`, on Streamable-HTTP POSTs, and requires servers to reject a request whose headers and body disagree (SEP-2243) [12]. A gateway can now make an allow, deny, or downgrade decision on the operation without parsing the body. The per-call boundary is a real enforcement point.

**Fleet-scalable.** List and read results now carry required cache metadata: a `ttlMs` freshness hint and a `cacheScope` of `public` or `private` (SEP-2549) [12]. With sessions gone, these results are stable enough to cache, which is what turns governing a fleet of agents from a per-agent tax into a per-server cost.

Set these five properties side by side and the shape is clear. An organization running agents on the 2026-07-28 spec gets, without extra work, a stream of actions that are attributable, typed,

traceable, individually enforceable, and cheap to observe at fleet scale. This is a genuine advance in what a protocol makes governable, and the temptation is to stop here and call the governance problem solved. It is not solved. A well-formed record has a limit, and that limit is the subject of the next section.

### 3 What the record cannot tell you

A record can be complete and still be silent about the thing that matters. Consider an agent action that is fully authorized, correctly typed, cleanly traced, and schema-valid, a record with nothing wrong in it, that opens a pull request introducing a SQL injection. Nothing in the protocol layer is equipped to notice, because nothing in the protocol layer looks at what the change does once it is merged. Veracode’s 2025 analysis found that 45% of AI-generated code samples introduced an OWASP Top-10 vulnerability, across more than a hundred models, eighty tasks, and four languages, with the rate roughly flat regardless of model size or recency [19]. The defective change and the correct change produce identical protocol records.

This is the clean-record trap, and the audit layer has always had it. A log tells you an action happened and was permitted; it never tells you the action was any good. The argument of this paper is that conditioning autonomy on task accuracy re-creates that same trap one level up. Grade an agent on how well it performs its task, how often its tool calls are correct and how rarely it is overridden, and a workflow that scores 99% on tool-call correctness still ships the Veracode defect in the remaining cases, because tool-call correctness and delivered-code quality are different measurements. Worse, an autonomy ladder built on task accuracy promotes a workflow precisely when its accuracy scores are high, which is the moment its output is drawing the least human scrutiny. The metric rises as the examination falls.

Task accuracy is also weak on its own terms. On structured agent benchmarks, current systems still fail roughly one in three attempts [18]. Weakness, though, is the smaller problem. Even a perfectly accurate agent would be graded on the wrong variable, because task accuracy measures the agent’s behavior and says nothing about the outcome that behavior delivered. Behavior-graded trust inherits every known failure of activity metrics. The variable that closes the loop lives one step past the tool boundary, at the delivered change, and no protocol call runs there.

### 4 The delivery evidence layer

Start with a defect that most engineering organizations already know they have. The delivery-metrics programs leaders run today, DORA’s four keys and DX Core 4, measure delivery at the team level, and they cannot tell AI-authored change apart from human-authored change. DX names this the attribution gap directly: the industry’s standard delivery metrics were designed before agents wrote code, and they aggregate over authorship [7]. An organization can watch its change-failure rate move and have no way to know whether its agents caused the move. The 95% with no measurable P&L return and the attribution gap are the same problem seen from two ends: an outcome that cannot be attributed to the work that produced it.

Closing that gap requires a layer the protocol does not provide, and naming it correctly matters, because an adjacent discipline already claimed a similar-sounding term for the opposite job. Context engineering builds a *context layer* whose purpose is to feed better inputs to agents so they act well [8]. The layer this paper describes runs in the other direction. Call it the *delivery evidence layer*: its purpose is to feed outcome evidence to the authorizer so the organization can decide what an agent is allowed to do next. Context feeds the actor. Evidence feeds the authorizer. It is one store, and it has four capabilities.

**Attribute.** Attribute joins the spec’s per-call action records to version-control and CI/CD events, so a delivered change carries a legible answer to which autonomous work produced it and through what path. The join is the implementer’s to build. The 2026-07-28 spec emits attributable, traceable action records (SEP-414, SEP-2575), and the organization’s own pipeline emits commits, reviews, and deploys, but nothing in the protocol stitches the two together. That stitch is the fix for the attribution gap, and it is the foundational capability; the other three depend on it.

**Evaluate.** Evaluate attaches outcome signals to each agent-authored change: change-failure rate, revert, rework, override-before-merge, whether the change held in production, and cost-to-outcome. These are delivery signals, not activity counts. They answer a different question than how many actions the agent took. They answer what the actions delivered, and what the delivery cost. Evaluate is where the conditioning variable of the whole argument, delivered outcome, actually gets computed.

**Serve.** Serve hands that evidence back as deterministic, typed, freshness-contracted context, delivered over the spec’s own substrate. Here the release pays a second dividend: full JSON Schema typing (SEP-2106) makes the served context checkable, required cache scopes and TTLs (SEP-2549) make its freshness a contract rather than a hope, and the per-call boundary (SEP-2567) is where it is delivered and re-authorized. The same protocol that emitted the events carries the answers back.

**Steer.** Steer feeds evaluations into boundary policy: the outcomes a workflow has actually delivered set the autonomy it is granted on its next call. Steer is the capability that turns the join into a loop, and it is where the protocol earns its place in this argument, because the per-call enforcement point is the actuator the loop writes to. It is also the capability most easily overstated, so the next section defines it carefully and states its limits first.

Attribute and Evaluate are the write path: they observe and record. Serve and Steer are the read path: they return evidence and act on it. None of the four is speculative. Attribute is a join over data an organization already has. Evaluate computes metrics teams already track. Serve is a typed read over a governed transport. Steer is policy at a boundary the spec now provides. What makes them a layer rather than four disconnected scripts is that they share one store of delivered-outcome evidence, keyed to the unit an organization actually wants to govern.

## 5 Earned autonomy

The 2026-07-28 release ships into a conformance-gated, SDK-tiered ecosystem: a conformance suite live since 2026-01-23, published SDK tiers since 2026-02-23, and, as of SEP-2484 (Final 2026-06-04), a standing rule that future Standards-Track SEPs cannot reach Final without a matching conformance scenario [14, 15]. The SDK tiers are the part worth studying. A Tier 1 SDK must pass 100% of the conformance suite; if any test fails continuously for four weeks, it is relegated to Tier 2; and features move through a lifecycle with a minimum twelve-month deprecation window measured from the revision release [13, 14]. Standing is earned by demonstrated conformance and lost publicly when the demonstration lapses.

That is a working model of earned standing, and it is a model humans run. No automated controller relegates an SDK. Maintainers publish tiers against measured conformance, on defined clocks, with a defined relegation rule, and the governance works. This is the precise shape to port. Earned autonomy is a rolling trust review in which workflow configurations hold published autonomy tiers, delivery-evidence-layer signals are the input, promotion and relegation follow defined triggers, and the resulting grant is enforced per call at the boundary the spec provides. Steer is that cadence.

It is not a controller that reads a metric and rewrites policy in a closed loop; it is a review rhythm with a clock and a rule, of the same kind the standard already runs on itself.

Every component of this cadence is mechanically expressible today. Identity and per-call enforcement come from the spec. Attribution, outcome evaluation, and served evidence come from the delivery evidence layer. Published tiers, clocks, and triggers are process. The only genuinely new input is the swap at the center: replace static scope grants, issued once and rarely revisited, with grants conditioned on delivered-outcome evidence. Automating the cadence, turning the rolling human review into a closed control loop, is the forward agenda, not a claim about today.

Stated plainly, and stated as the sentence a skeptic should attack: autonomy conditioned on task accuracy re-creates at the trust layer the same clean-record trap the audit layer already has; delivered outcome is the only conditioning variable that closes the loop, and the 2026-07-28 spec is the first standard substrate on which that condition is enforceable per action.

**The failure modes are dials, not objections.** Because Steer is a cadence, its known failure modes are design parameters that set the cadence’s dials. *Attribution lag* — outcomes arrive days or weeks after the change — sets the length of the trailing window the review scores on; instantaneous scoring is the mistake the window length corrects. *Unstable identity* — agent instances are ephemeral and interchangeable — sets the unit of trust: the durable thing to govern is a workflow configuration, not an instance, because the configuration is what produced the work and what will produce it again. *Goodhart’s law* sets the gating signal: gate on held-in-production outcomes and never on activity counts, because the moment the metric is a throughput count it is gamed. *Small-n* — too few observations to trust — sets the cold-start default: absent evidence, a workflow starts in a sandbox tier, since autonomy is earned from evidence and the safe default without it is low.

**Two clarifications keep the framework honest** about what the protocol does and does not provide. First, the per-call boundary is an enforcement point, not a scope engine. The spec gives the point at which authorization is checked on every call; it does not decide what a given principal may do. The scope semantics, the policy itself, remain the implementer’s design. Authentication is not authorization scope. Second, per-call identity attributes to the authenticated principal and the named client implementation, carried in `clientInfo`, and not to an individual agent or subagent. Agent-level attribution is a convention an organization builds on top, and it is exactly why the unit of trust is a workflow configuration: that is the granularity the evidence layer can attribute to, not a per-instance identity the protocol never reported.

This shape has precedent outside software delivery. Open-source foundations already promote projects by earned standing: the CNCF moves a project from Sandbox to Incubating to Graduated on demonstrated adoption, security review, and governance maturity [4]. The criteria are partly quantitative and partly qualitative, and the decision is ratified by humans. It is evidence-based maturation with human ratification, not a purely metric-gated switch, which is exactly the posture earned autonomy takes toward a workflow rather than a whole project.

There is also direct empirical support for building the feedback capability the framework depends on. DORA’s 2025 research found that AI acts as an amplifier: it has a positive relationship with delivery throughput and a negative one with stability, and which effect dominates is conditional on a team’s existing capability [6]. The organizational property that decides whether AI speed helps or hurts is the delivery-feedback capability, which is the same capability this paper argues an organization must build. The amplifier result is not only a stake. It is corroboration that the conditioning variable is the right one.

The cadence is configuration, not research. A gateway can express the entire conditioning rule in

```

# Grant repo-write only while a workflow configuration's
# agent-authored changes keep holding in production.
policy conditional_write_scope {
  when    request.scope == "repo:write"
  allow    identity.workflow_tier == "trusted"
          && evidence(identity.workflow_id).revert_rate_28d < 0.05
  else     downgrade "repo:read"    # relegation, enforced per call
}

```

Figure 1: **Illustrative.** A gateway policy conditioning a write-scope grant on a workflow configuration’s trailing 28-day revert rate. The evidence lookup is the delivery evidence layer; the trailing window absorbs attribution lag; the unit of trust is a workflow configuration, not an instance; and the downgrade is relegation enforced at the per-call boundary. This is the same shape teams already use to gate a deploy, pointed at a different signal.

a handful of lines: a scope grant that holds only while a workflow configuration’s trailing outcome evidence clears a threshold, and that downgrades at the same per-call boundary when it does not.

## 6 Recommendations

The move a leader can make on Monday costs no procurement. It is to stand up the cadence as a process before it is a product. Concretely: publish autonomy tiers for the workflow configurations already running agents, from a sandbox tier up through a trusted tier; define the trailing window and the outcome signals each tier is scored on; set the relegation triggers and the clock, borrowing the standard’s own four-week shape as a starting point; and default every new or sparsely-observed workflow to the sandbox. This is a review rhythm a platform or developer-experience team can run in a spreadsheet on the first day and encode in gateway policy as it matures. None of it requires buying anything.

The capability to build alongside it is the delivery evidence layer, and it belongs in the same budget cycle as the MCP boundary work, not after it. The boundary makes actions enforceable; the layer makes outcomes legible; the cadence needs both. Sequencing the enforcement work first and the evidence work later produces the exact state this paper opened on: a fleet whose actions are perfectly recorded and whose value is unattributable.

When evaluating any system, internal or purchased, that claims to govern agent autonomy, five questions separate an outcome-conditioned tool from an activity-graded one:

- **Attribute at the change level.** Does it join governed actions to the commit, review, and deploy they produced, or does it stop at the action record?
- **Close the governed-action-to-shipped-result loop.** Can a granted scope be traced to the outcome it caused, so authority and result meet in one place?
- **Segment authorship.** Does it tell agent-authored change apart from human-authored change, so the attribution gap is actually closed rather than aggregated over?
- **Resist adoption proxies.** Does it condition trust on held-in-production outcomes and refuse activity and adoption counts, such as action volume or acceptance rate, as stand-ins for delivery?
- **Serve context over governed rails.** Does it return evidence to the agents over the same typed, cache-contracted, per-call-authorized transport the actions ran on?

The 2026-07-28 spec settles the question of whether an organization can see and control what its

agents do, action by action. It leaves open the question this paper is about: whether the organization can tell what those actions delivered, and condition trust on the answer. The substrate for that second question now exists. The layer above it, and the cadence above that, are the work in front of every team running agents at scale.

## References

- [1] Agent platform vendors (industry). Progressive autonomy controls. Representative industry usage; see e.g. <https://www.mindstudio.ai/> and <https://mightybot.ai/>, 2025. Autonomy tiers gated on agent task performance.
- [2] Peter Alexander and Model Context Protocol Maintainers. SEP-2567: Sessionless MCP via explicit state handles. <https://modelcontextprotocol.io/seps/2567-sessionless-mcp>, 2026.
- [3] Amazon Web Services. Security principles for agentic AI. <https://aws.amazon.com/>, 2025.
- [4] Cloud Native Computing Foundation. Project lifecycle: Sandbox, incubating, graduated. <https://contribute.cncf.io/projects/lifecycle/>, 2025.
- [5] Digital Apprentice authors. The digital apprentice: Progressive autonomy for AI agents. <https://arxiv.org/abs/2606.04321>, 2026. arXiv preprint; author list and title to confirm at publication.
- [6] DORA (Google Cloud). State of AI-assisted software development 2025. <https://dora.dev/research/2025/>, 2025.
- [7] DX. The DX core 4 and the attribution gap in AI-authored change. <https://getdx.com/>, 2025.
- [8] IBM. The structured context layer for enterprise AI. <https://www.ibm.com/think>, 2025.
- [9] McKinsey & Company. Trust in the age of agents. <https://www.mckinsey.com/>, 2025.
- [10] MIT Media Lab (Project NANDA). The GenAI divide: State of AI in business 2025. MIT Media Lab, Project NANDA report, 2025. 300 enterprise GenAI initiatives, 52 interviews, 153 surveys. Canonical source URL to confirm at publication.
- [11] Model Context Protocol Maintainers. Deprecated features registry (2026-07-28 revision). <https://modelcontextprotocol.io/specification/draft/deprecated>, 2026. SEP-2577: Roots, Sampling, Logging deprecated.
- [12] Model Context Protocol Maintainers. Specification draft changelog (2026-07-28 revision). <https://modelcontextprotocol.io/specification/draft/changelog>, 2026. SEP-2575, SEP-2468, SEP-2106, SEP-414, SEP-2243, SEP-2549.
- [13] Model Context Protocol Maintainers. Feature lifecycle and deprecation policy (SEP-2596). <https://modelcontextprotocol.io/community/feature-lifecycle>, 2026.
- [14] Model Context Protocol Maintainers. SDK tiers and conformance. <https://modelcontextprotocol.io/community/sdk-tiers>, 2026.

- [15] Model Context Protocol Maintainers. SEP-2484: Conformance scenario required for standards-track Final. <https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2484>, 2026. Final 2026-06-04.
- [16] Matthias Roder. The earned autonomy gradient. Industry framework. Named prior-art framework for agent autonomy graded on task performance. Canonical citation (year, URL) to confirm at publication.
- [17] Stanford HAI. The AI index report 2025. <https://hai.stanford.edu/ai-index/2025>, 2025.
- [18] Stanford HAI. The AI index report 2026. <https://hai.stanford.edu/ai-index/2026>, 2026.
- [19] Veracode. 2025 GenAI code security report. <https://www.veracode.com/blog/genai-code-security-report>, 2025.